



Class 10 — Angular-Directives

Class 10 — Angular Directives

Table of Contents

- ☐ [What is ng-template?](#)
 - ☐ [Instantiate a Template with ngTemplateOutlet](#)
 - ☐ [Pass Data into a Template — Context](#)
 - ☐ [What is a Directive?](#)
 - ☐ [Attribute Directive — bookHighlight](#)
 - ☐ [exportAs — Expose a Directive to the Template](#)
 - ☐ [The * Syntax — Desugaring](#)
 - ☐ [Structural Directive — bookShow](#)
 - ☐ [Interview Questions](#)
-

1. What is ng-template?

An `ng-template` is a **blueprint of HTML** that Angular holds in memory but **never renders on its own**.

Think of it like a cookie cutter — it defines a shape, but nothing appears on the page until something explicitly stamps it out.

Rule

`ng-template` produces **zero DOM nodes** unless something explicitly instantiates it.

Demo — Add this anywhere in `book-list.html`

```
<!-- This renders NOTHING in the browser -->
<ng-template>
```

```
<p>I am a blueprint. I am invisible.</p>
</ng-template>
```

Open DevTools → Elements tab → the `<p>` is completely absent from the DOM.

Why does ng-template exist?

Angular's structural directives (`*ngIf`, `*ngFor`, and any custom ones you build) need a container to hold "a piece of HTML that may or may not appear in the DOM". That container is always an `ng-template` — Angular wraps your element in one automatically when you use the `*` shorthand syntax.

2. Instantiate a Template with ngTemplateOutlet

To actually **render** an `ng-template`, you use the `NgTemplateOutlet` directive.

Step 1 — Give the template a reference variable

```
<ng-template #loadingTpl>
  <tr>
    <td colspan="4" class="center">Loading books...</td>
  </tr>
</ng-template>
```

`#loadingTpl` is a **template reference variable**. Angular stores the `TemplateRef` object under this name.

Step 2 — Render it with NgTemplateOutlet

```
<ng-container *ngTemplateOutlet="loadingTpl"></ng-container>
```

`ng-container` is a **ghost wrapper** — it exists only for Angular's structural use and never appears in the DOM.

`*ngTemplateOutlet` tells Angular: "stamp out the blueprint stored in `loadingTpl` right here."

Step 3 — Import NgTemplateOutlet in `book-list.ts`

```
import { NgTemplateOutlet, DatePipe } from '@angular/common';

@Component({
  selector: 'app-book-list',
  imports: [RouterLink, DatePipe, NgTemplateOutlet],
  templateUrl: './book-list.html',
  styleUrls: ['./book-list.scss'],
})
export class BookList implements OnInit {
  books: Book[] = [];
  isLoading = true;
}
```

Step 4 — Show loading state vs content

```
@if (isLoading) {
  <ng-container *ngTemplateOutlet="loadingTpl"></ng-container>
} @else {
  <!-- table rows -->
}

<ng-template #loadingTpl>
  <tr>
    <td colspan="4" class="center">Loading books...</td>
  </tr>
</ng-template>
```

Set `isLoading = false` inside the `subscribe` callback — watch the UI switch instantly.

3. Pass Data into a Template — Context

A template blueprint is static by default. But you can **inject data into it** at the moment you stamp it out using a **context object**.

The \$implicit key

`$implicit` is the **default variable** in a context object.
Inside the template, `let-varName` (no `=` sign) binds to it automatically.

Named keys

Any other key in the context object requires `let-varName="keyName"` to bind explicitly.

Demo — Typed reusable book row in `book-list`

Define the template in `book-list.html`

```
<ng-template #bookRowTpl let-book let-i="index">
  <tr>
    <td>{{ i + 1 }}</td>
    <td>{{ book.name }}</td>
    <td>{{ book.author }}</td>
    <td>{{ book.rating }} ★</td>
  </tr>
</ng-template>
```

Stamp it out with context for every book

```
@for (book of topRatedBooks; track book.id; let i = $index)
{
  <ng-container *ngTemplateOutlet="bookRowTpl; context: {
    $implicit: book, index: i }">
  </ng-container>
}
```

The mental model to lock in

This is **exactly** how `*ngFor` exposes `index`, `first`, `last`, `even`, `odd` to your template.

Every named key in the context object becomes a bindable variable via `let-varName="keyName"`.

You will use this exact same mechanism when building structural directives.

4. What is a Directive?

A **directive** is a class that adds behaviour to a DOM element.

Three Types of Directives

Type	What it does	Real examples
Component	Renders its own UI	<code><app-book-list></code> , <code><book-header></code>
Attribute	Changes appearance or behaviour of an existing element	<code>ngClass</code> , <code>ngModel</code> , <code>routerLinkActive</code>
Structural	Adds or removes DOM elements	<code>*ngIf</code> , <code>*ngFor</code>

5. Attribute Directive — Highlight

An attribute directive that colors a book row based on its rating.

`src/app/directive/highlight.t`

```
import { Directive, ElementRef, Input, OnInit } from '@angular/core';

@Directive({
  selector: '[bookHighLight]',
  host: {

  }
})
export class Highlight implements OnInit {
  @Input() rating = 0;

  constructor(private el: ElementRef<HTMLElement>) {}

  ngOnInit(): void {
    this.highlight(this.rating);
  }

  private highlight(rating: number): void {
    if (this.rating >= 5) {
      this.el.nativeElement.style.color = '#2e7d32';
    }
  }
}
```

```

        this.el.nativeElement.style.fontWeight = 'bold';
    } else if (this.rating >= 4) {
        this.el.nativeElement.style.color = '#e65100';
        this.el.nativeElement.style.fontWeight = 'normal';
    }
}
}

```

How it is used in `book-list.html`

```

<!-- On each book row – rating passed from the book object
-->
<ng-template #bookRow let-book let-i="index">
  <tr bookHighLight [rating]="book.rating"
    [routerLink]="['/books/edit/reactive', book.id]">
    <td>{{ i + 1 }}</td>
    <td>{{ book.name }}</td>
    <td>{{ book.author }}</td>
    <td>{{ book.category }}</td>
    <td>{{ book.rating }}</td>
  </tr>
</ng-template>

```

Breakdown — every piece explained

selector: '[bookHighLight]' Square brackets mean CSS attribute selector — matches any element that carries the `bookHighLight` attribute. Not just one element type, any element.

@Input() rating = 0 Receives the rating value from the template via `[rating]="book.rating"`. Default is `0`

constructor(private el: ElementRef<HTMLElement>) Standard Angular DI — `ElementRef` wraps the native DOM node the directive is sitting on. `el.nativeElement` gives direct access to it.

ngOnInit(): void Lifecycle hook — fires once after Angular has set all `@Input()` values. This is the right place to apply initial styling based on inputs, because `@Input()` values are not available in the constructor yet.

`this.el.nativeElement.style.color = '#2e7d32'` Direct DOM write via `ElementRef`.
Simple and works.

Observe in the browser

Import and add `Highlight` to `book-list.ts` imports, then load the page:

- Rows with `rating >= 5` → green text
- Rows with `rating >= 4` → orange text
- Rows with `rating < 4` → no change (directive does nothing for them yet)

```
import { HighlightDirective } from '../directives/highlight.directive';

@Component({
  imports: [RouterLink, DatePipe, NgTemplateOutlet, HighlightDirective],
})
```

Observe in the browser

Hover over any row → background turns blue. Move away → background clears. Top-rated rows get the yellow color you passed in.

Passing \$event — handling click with coordinates

To receive the native DOM event object, pass `$event` in the `host` string:

```
host: {
  '(mouseenter)': 'onMouseEnter()',
  '(mouseleave)': 'onMouseLeave()',
  '(click)': 'onRowClick($event)',
},
```

```
onRowClick(event: MouseEvent): void {
  console.log('Clicked at:', event.clientX, event.clientY);
}
```

For keyboard events:

```

host: {
  '(keydown.enter)': 'onEnterKey($event)',
},

onEnterKey(event: KeyboardEvent): void {
  console.log('Enter pressed on row');
}

```

7. exportAs — Expose a Directive to the Template

`exportAs` lets you give a directive a **public name** so a template can grab a reference to the directive **instance** itself using a template reference variable.

The syntax

```

@Directive({
  selector: '[appHighlight]',
  exportAs: 'appHighlight', // name this directive
  standalone: true,
  host: { ... },
})

```

```

<tr appHighlight #hl="appHighlight">
  <!-- hl is now a typed HighlightDirective instance -->
  <td>Hovered: {{ hl.isHighlighted }}</td>
</tr>

```

`#hl="appHighlight"` — the string `"appHighlight"` must match the `exportAs` value. `hl` gives you direct access to every public property and method on `HighlightDirective`.

In Class 8 template-driven forms we see:

```

<form #form="ngForm">
  <input #nameField="ngModel" name="name" ngModel/>
</form>

```

- `ngForm` → the `exportAs` value of Angular's `NgForm` directive

- `ngModel` → the `exportAs` value of Angular's `NgModel` directive

`form.valid`, `form.submitted`, `nameField.touched`, `nameField.errors` — all of these read properties directly off the directive instance. You were using `exportAs` without knowing it.

Demo — Add exportAs to HighlightDirective

```
import { Directive, ElementRef, inject, input } from '@angular/core';

@Directive({
  selector: '[appHighlight]',
  exportAs: 'appHighlight',
  standalone: true,
  host: {
    '(mouseenter)': 'onMouseEnter()',
    '(mouseleave)': 'onMouseLeave()',
  },
})
export class HighlightDirective {
  appHighlight = input('#eef6ff');
  defaultColor = input('');

  isHighlighted = false; // public – template can read this

  private el = inject(ElementRef);

  onMouseEnter(): void {
    this.isHighlighted = true;
    this.highlight(this.appHighlight() || this.defaultColor() || '#eef6ff');
  }

  onMouseLeave(): void {
    this.isHighlighted = false;
    this.highlight('');
  }
}
```

```
// Public method – callable from the template
forceHighlight(): void {
  this.isHighlighted = true;
  this.highlight(this.appHighlight() || '#eef6ff');
}

private highlight(color: string): void {
  this.el.nativeElement.style.backgroundColor = color;
}
}
```

Use exportAs in `book-list.html`

```
@for (book of books; track book.id) {
  <tr [appHighlight]="'#eef6ff'" #hl="appHighlight"
    [routerLink]="['/books/edit/reactive', book.id]>
    <td>{{ book.name }}</td>
    <td>{{ book.author }}</td>
    <td>{{ book.rating }} ★</td>
    <td>
      <span>{{ hl.isHighlighted ? 'Hovered' : '' }}</span>
    </td>
  </tr>
}
```

Call a directive method from a button

```
<button (click)="hl.forceHighlight()">Highlight this row</button>
<tr [appHighlight]="'#fff3cd'" #hl="appHighlight">
  <td>{{ book.name }}</td>
</tr>
```

`#hl` is scoped to the current template block. Inside `@for`, each iteration has its own `hl` instance.

What you get with vs without exportAs

Template variable	What you get
<code>#myRow</code> (no exportAs)	<code>HTMLTableRowElement</code> — the raw DOM node
<code>#hl="appHighlight"</code>	<code>HighlightDirective</code> instance — your TypeScript class
<code>#form="ngForm"</code>	Angular's <code>NgForm</code> directive instance
<code>#field="ngModel"</code>	Angular's <code>NgModel</code> directive instance

8. The * Syntax — Desugaring

Before building structural directives, you must understand what `*` actually means in Angular.

The Rule

The asterisk `*` is **syntactic sugar**. Angular compiles any `*directive="expr"` into an `<ng-template>` with a property binding.

Side-by-side

```
<!-- What you write (shorthand) -->
<tr *appShowIf="isAdmin">
  <td>Secret Row</td>
</tr>

<!-- What Angular compiles it to (longhand) -->
<ng-template [appShowIf]="isAdmin">
  <tr>
    <td>Secret Row</td>
  </tr>
</ng-template>
```

The `<tr>` is wrapped in an `ng-template`. The directive selector becomes a property binding on that template. Both forms are identical — the `*` version is just shorter to write.

What Angular injects into a structural directive

Injectable	Type	What it is
<code>TemplateRef</code>	Blueprint	The <code>ng-template</code> wrapping your element
<code>ViewContainerRef</code>	Slot	The anchor in the DOM where stamped views are inserted

With modern Angular you get both via `inject()`:

```
private templateRef = inject(TemplateRef);
private vcr         = inject(ViewContainerRef);
```

The two key ViewContainerRef methods

```
this.vcr.createEmbeddedView(templateRef, context?) // stamp
s the template into the DOM
this.vcr.clear()                                // remov
es all stamped views
```

Microsyntax — multiple inputs in a structural directive

```
<tr *appMinRating="4; of: book">
```

The `; of: book` part passes a second input. Angular desugars this as:

```
<ng-template [appMinRating]="4" [appMinRatingOf]="book">
  <tr></tr>
</ng-template>
```

Rule: `of:` in microsyntax maps to `[directiveSelector + 'Of']` → `appMinRatingOf`.

This is the same rule `*ngFor="let item of items"` uses — `of` maps to `ngForOf`.

9. Structural Directive — Show

A custom `*ngIf` clone. This is the actual directive built during class in `src/app/directive/show.ts`.

Show directive — code written in class

```

import { Directive, Input, OnInit, TemplateRef, ViewContainerRef } from '@angular/core';

export interface TemplateContext {
  $implicit: string
}

@Directive({
  selector: '[bookShow]',
})
export class Show implements OnInit {
  @Input() bookShow!: boolean;

  constructor(private vcr: ViewContainerRef, private tpr: TemplateRef<TemplateContext>) {}

  ngOnInit(): void {
    this.render(this.bookShow);
  }

  render(isShow: boolean) {
    if (isShow) {
      this.vcr.createEmbeddedView(this.tpr);
    } else {
      this.vcr.clear();
    }
  }
}

```

Show directive — usage in book-list.html

```

<ng-template [bookShow]="false">
  <table>
    <thead>
      <ng-container *ngTemplateOutlet="bookHeader"></ng-container>
    </thead>

```

```

    <tbody>
      @for (book of topRatedBooks; track book.id; let i =
$index) {
        <ng-container
          *ngTemplateOutlet="bookRow; context: { $implicit:
book, index: i }"
        ></ng-container>
      }
    </tbody>
  </table>
</ng-template>

```

`[bookShow]="false"` → the table is hidden. Change to `[bookShow]="true"` → the table appears.

`*bookShow="false"` desugars to:

```

<ng-template [bookShow]="false">
  <!-- element here -->
</ng-template>

```

That is exactly what you wrote in `book-list.html`. You used the longhand form directly — which means you already understand what `*` compiles to. The shorthand version on a real element would look like:

```

<!-- shorthand – * on the element -->
<table *bookShow="true">
  ...
</table>

<!-- Angular compiles it to exactly the longhand you wrote
-->
<ng-template [bookShow]="true">
  <table>...</table>
</ng-template>

```

Both are identical. The `*` form is just shorter.

Show directive — line by line

`selector: '[bookShow]'` Matches any element or `ng-template` that has the `bookShow` attribute. Because the input name `bookShow` matches the selector, the shorthand `*bookShow="condition"` automatically binds `condition` to `this.bookShow`.

`@Input() bookShow!: boolean` The `!` is a TypeScript non-null assertion — tells the compiler "trust me, this will be set before use." Angular sets it via the property binding `[bookShow]="false"` before `ngOnInit` fires.

`constructor(private vcr: ViewContainerRef, private tpr: TemplateRef<TemplateType>)` Angular injects both automatically for any structural directive:

- `ViewContainerRef` — the anchor point in the DOM where stamped content is inserted
- `TemplateRef` — the blueprint (`ng-template`) holding the HTML to stamp out

`ngOnInit(): void { this.render(this.bookShow) }` Fires once after inputs are set. Reads `this.bookShow` and stamps or clears the template.

`this.vcr.createEmbeddedView(this.tpr)` Takes the `TemplateRef` blueprint and stamps a real DOM copy of it at the `ViewContainerRef` anchor. This is what makes the content appear.

`this.vcr.clear()` Removes all previously stamped views from the `ViewContainerRef`. This is what makes the content disappear.

What createEmbeddedView and clear() do

`vcr.createEmbeddedView(tpr)`

1. Takes the `ng-template` blueprint (`tpr`)
2. Creates a real DOM copy of its entire contents
3. Inserts that copy at the `ViewContainerRef` anchor — exactly where the directive was placed in the HTML
4. The content is now live in the page

`vcr.clear()`

1. Finds all views currently stamped at this `ViewContainerRef`
2. Destroys them and removes them from the DOM completely
3. The content is gone — not hidden with CSS, actually absent from the DOM tree

Important — `ngOnInit` works here, but only for a static condition

Because `ngOnInit` fires once, `render()` runs once with the initial value of `bookShow`.

Scenario	Works with <code>ngOnInit</code> ?
<code>[bookShow]="false"</code> (hardcoded)	Yes — value never changes
<code>[bookShow]="isAdmin"</code> where <code>isAdmin</code> can change	No — <code>ngOnInit</code> won't re-run

For a **reactive** directive that responds to a changing condition (the real `*ngIf` behaviour), use an `@Input()` setter instead:

```
@Input()  
set bookShow(value: boolean) {  
  this.render(value); // fires every time the value changes  
}
```

A setter runs every time Angular writes a new value to that input — no lifecycle hook needed.

10. Interview Questions

Q1. What does the `*` (asterisk) mean in `*ngIf` or `*bookShow`?

It is syntactic sugar. Angular compiles `*bookShow="isAdmin"` into:

```
<ng-template [bookShow]="isAdmin">  
  <!-- original element -->  
</ng-template>
```

The directive receives the `ng-template` as a `TemplateRef` via DI. The `*` is just a shorter way to write the longhand form — both compile to identical output.

Q2. What is `TemplateRef` and what is `ViewContainerRef`? Why do structural directives need both?

- `TemplateRef<C>` — a reference to an `ng-template` (the blueprint). `C` is the context type.
- `ViewContainerRef` — an anchor point in the DOM where views are inserted.

A structural directive needs both: `TemplateRef` provides the HTML to stamp; `ViewContainerRef` provides the location to insert it. Without `ViewContainerRef` you have a blueprint with nowhere to place it.

Q3. What is `ng-template` ? Does it render anything in the browser?

`ng-template` is a blueprint of HTML that Angular keeps in memory but never inserts into the DOM on its own. In production builds it leaves no trace — not even a comment node. It is only rendered when explicitly instantiated via `ngTemplateOutlet` or a structural directive's `ViewContainerRef.createEmbeddedView`.

Q4. What is `$implicit` in a template context?

`$implicit` is the **default binding key** in a context object. Inside a template, `let-book` (no `= "keyName"`) automatically binds to `$implicit`. All other keys require `let-varName="keyName"`.

This is how `*ngFor="let item of items"` works — `NgFor` passes each iteration item as `$implicit` in the context; `let item` picks it up.

Q5. What is `exportAs` ? Give a real example from the Book-App and from Angular's built-ins.

`exportAs` gives a directive a name so the template can access the directive instance via `#ref="exportAsName"`:

```
@Directive({ selector: '[appHighlight]', exportAs: 'appHighlight' })
```

```
<tr appHighlight #hl="appHighlight">
  {{ hl.isHighlighted }} <!-- reads a property on HighlightDirective -->
</tr>
```

Angular built-in examples:

- `NgForm` → `exportAs: 'ngForm'` → used as `#form="ngForm"` in template-driven forms (Class 8)
- `NgModel` → `exportAs: 'ngModel'` → used as `#field="ngModel"` to read `field.errors`, `field.touched`

Without `exportAs`, `#myRow` gives you the raw `HTMLTableRowElement`. With it, you get the typed directive instance.